

RELATÓRIO TÉCNICO DE REFLEXÃO ARQUITETURAL – PROJETO SAGSA-LITE

Curso: Técnico em Desenvolvimento de Sistemas

Turma: APTEC132T26 (Módulo III)

Unidade Operacional: Centro de Formação Profissional SENAI Plínio Gilberto Kröeff

Nomes: Andrei e Yuri

1. Respostas às Questões Avaliativas

- **Questão A:** Foi necessário alterar alguma linha de código dentro do pacote `sagsa.core` para implementar a mudança de banco? Justifique o porquê com base no conceito de acoplamento.

Resposta: Não, nenhuma linha de código ou classe pertencente ao pacote `sagsa.core` precisou ser alterada ou adaptada para que o sistema passasse a consumir dados da Nuvem SQL.

Justificativa Técnica: Isso foi possível porque a arquitetura do SAGSA-Lite foi projetada seguindo o pilar do **Acoplamento Fraco (Loose Coupling)**. O motor de regras (`GeradorEstrategiaCore`) não conhece e não depende de nenhuma implementação concreta de banco de dados (seja ela Excel ou SQL). O core depende exclusivamente de uma abstração: a interface `RepositoryContrato`. Como ambas as classes de dados (`FakeExcelRepository` e `BancoDadosRealRepository`) assinam e respeitam esse mesmo contrato formal, o motor de regras permanece completamente isolado e protegido contra alterações de infraestrutura tecnológica.

Questão B: Quais seriam as consequências para o cronograma do projeto se os dados de consulta estivessem programados diretamente dentro do método de exibição na View?

Resposta da Squad: Se a lógica de acesso aos dados estivesse programada diretamente dentro dos métodos de exibição da View, o projeto sofreria de **Baixa Coesão e Acoplamento Forte**, gerando o antipadrão conhecido como "Código Espaguete".

Justificativa Técnica: Teria grandes consequências para o cronograma, resultando em atrasos por retrabalho. Qualquer alteração na origem dos dados (como a mudança de planilhas para a nuvem pedida pelo cliente) exigiria uma refatoração direta nos arquivos de interface visual. Isso quebraria o isolamento lógico, gerando um "Efeito Dominó" onde alterações na infraestrutura causaram bugs visuais colaterais. Além disso, impediria o trabalho em paralelo da equipe já que o desenvolvedor de interface estaria mexendo no mesmo arquivo que o desenvolvedor de banco e inviabilizaria a execução de testes unitários isolados, estendendo drasticamente o tempo necessário para homologar a entrega da Sprint.